

Fast Tournament Equity Computation via Generating-Function Quadrature and FFT-Accelerated Subproduct Trees

Sam Rosenstrauch
sarose550@gmail.com

July 21, 2026

Abstract

The Independent Chip Model (ICM) maps poker tournament chip stacks to expected prize money, but exact computation requires summing over all $n!$ elimination orderings. I present a deterministic algorithm that reformulates ICM equity as a one-dimensional integral over generating-function coefficients, evaluated by Gaussian quadrature ($Q = 256$ nodes, relative error $< 5 \times 10^{-12}$). The central challenge—computing leave-one-out polynomial products for all n players—is solved by an FFT-accelerated subproduct tree whose propagation phase is the adjoint of its build phase, reducing cost from $\mathcal{O}(nk)$ to $\mathcal{O}(n \log^2 k)$ per quadrature point. A roofline cost model with harmonic-mean bandwidth blending dispatches automatically between a batched linear engine and a hybrid block-tree engine. Implementations for Apple M3 Pro, AMD Zen 4 (with AOCL-FFTW), and NVIDIA B200 GPU are evaluated: the GPU computes 6.3 million player equities ($k = 100$) in 626 ms, and all 262,144 equities at $k = n$ in 117.9 ms.

1 Introduction

Multi-table poker tournaments are structured competitions in which players begin with equal chip stacks and are progressively eliminated as they lose their chips. A payout structure $(\pi_1, \pi_2, \dots, \pi_k)$ specifies the prize awarded to each finishing position, with $k \leq n$ positions receiving nonzero prizes and $\pi_1 \geq \pi_2 \geq \dots \geq \pi_k \geq 0$. The *Independent Chip Model* (ICM) provides a principled mapping from a player’s chip stack to their expected tournament equity—the expected prize money they will receive [Malmuth, 2001, Harville, 1973].

Despite its central role in tournament strategy, exact ICM computation is fundamentally difficult because the number of possible elimination orderings grows factorially in n . For $n = 20$ there are 2.4×10^{18} orderings to consider; for $n = 30$, the count reaches 2.7×10^{32} . Bitmask dynamic programming reduces this to $\mathcal{O}(n \cdot 2^n)$ time by iterating over subsets of surviving players [Malmuth, 2001], but the exponential state space limits practical computation to roughly $n \leq 23$. Monte Carlo sampling provides approximate solutions for larger fields [ICMIZER, 2024], but error shrinks only as $\mathcal{O}(1/\sqrt{N})$, making high-precision results prohibitively expensive.

The key observation that enables a better approach is that each player’s ICM equity can be expressed as a one-dimensional integral rather than an exponential sum. This follows from the *exponential clock framing*, where each player is assigned an independent exponential random variable and players are eliminated in order of increasing firing time. Under this framing, the combinatorial sum collapses into an integral that depends only on the chip stacks.

The integral is evaluated by Gaussian quadrature with $Q = 256$ nodes, achieving exponential convergence to double-precision accuracy. The remaining challenge is evaluating the integrand efficiently at each quadrature point, which requires computing a degree- k leave-one-out polynomial product for each of n players. I address this with an FFT-accelerated binary subproduct tree whose propagation phase is the adjoint of the build phase—a cross-correlation that distributes the payout vector to all leaves simultaneously.

My contributions are:

1. A rigorous derivation of ICM equities as coefficient extractions from a generating function, evaluated via numerical quadrature (Sections 3–4).
2. A hybrid engine combining a subproduct tree for inter-block computation with direct polynomial arithmetic for intra-block computation, with correctness proved via the adjoint relationship between convolution and cross-correlation (Section 4).
3. A roofline cost model with harmonic-mean bandwidth blending that achieves 42/44 correct engine dispatch decisions on AMD Zen 4, replacing simpler FMA-counting models (Section 4.5).
4. Platform-specific implementations for ARM64 (Apple M3 Pro), x86-64 (AMD Zen 4 with AOCL-FFTW), and CUDA (NVIDIA B200 GPU), demonstrating the approach at scale (Sections 6 and 7).

2 Related Work

ICM and the Malmuth–Harville model. The probability model underlying ICM was introduced by Harville [1973] for horse-racing handicapping and adapted to poker tournaments by Malmuth [2001]. Under this model, the probability that any surviving player is eliminated next is proportional to their chip stack. The standard exact algorithm uses bitmask dynamic programming over all 2^n subsets of surviving players, running in $\mathcal{O}(n \cdot 2^n)$ time. For large fields, Monte Carlo sampling is the prevailing approach, with error converging as $\mathcal{O}(1/\sqrt{N})$.

Exponential clock formulation. The equivalence between the Malmuth–Harville elimination process and independent exponential clocks was noted by Tysen Streib in a 2012 forum post, who used it to develop an efficient Monte Carlo sampling method. The key insight is that assigning each player an $\text{Exp}(S_j)$ random variable and sorting by realized value produces an elimination order whose distribution matches the sequential-elimination model exactly, while requiring only n random draws and one sort per sampled tournament.

Polynomial arithmetic and subproduct trees. The subproduct tree (also called a *subresultant tree* or *product tree*) is a standard tool in computer algebra for multi-point evaluation and interpolation. von zur Gathen and Gerhard [2013] provide a comprehensive treatment, including the $\mathcal{O}(n \log^2 n)$ algorithms for computing the product of n linear factors and evaluating a polynomial at n points. My use of the tree is closest to the “accumulating remainder” formulation, but with the critical difference that propagation computes cross-correlations (the adjoint of convolution) rather than polynomial remainders. This adjoint perspective, proved in Theorem 4.4, appears to be new in this context.

FFT libraries and automatic tuning. FFTW [Frigo and Johnson, 2005] introduced the idea of empirical auto-tuning for FFT plans, generating machine-specific “codelets” at install time. AMD’s AOCL-FFTW extends this with 636 AVX-512-specific codelets (versus one in the upstream distribution), delivering $1.2\text{--}2\times$ speedups at key sizes on Zen 4 architectures [Advanced Micro Devices, Inc., 2024]. I use dual-library dispatch (AOCL-FFTW vs. Intel MKL, selected per-size by offline calibration) on x86-64. The broader philosophy of empirically-tuned numerical libraries traces to ATLAS [Whaley and Dongarra, 1998] and SPIRAL [Püschel et al., 2005].

Roofline performance models. The roofline model of Williams et al. [2009] bounds achievable performance by the minimum of peak compute and memory bandwidth, parameterized by arithmetic intensity. My dispatch cost model extends this idea to multi-level cache hierarchies using harmonic-mean bandwidth blending—a natural consequence of modeling total transfer time as the sum of time spent at each cache level.

GPU FFT and CUDA Graphs. NVIDIA’s cuFFT library supports batched FFT execution, and the cuFFTDx extensions [NVIDIA Corporation, 2024b] enable fused device-side FFTs that eliminate intermediate memory round-trips. CUDA Graphs [NVIDIA Corporation, 2024a] amortize kernel launch overhead by capturing and replaying entire execution graphs, which is essential for the 40+ kernel launches in a single tree traversal.

3 Mathematical Derivation

This section derives the integral representation that is the foundation of the algorithm. All of the key ideas—the exponential clock, the generating function, and the one-dimensional integral—are presented here, with supporting proofs where they provide insight.

3.1 The Malmuth–Harville Model and Exponential Clocks

Definition 3.1 (Tournament instance). *A tournament instance is a tuple $(n, \mathbf{S}, \boldsymbol{\pi})$ where $n \in \mathbb{N}$ is the number of players, $\mathbf{S} = (S_1, \dots, S_n) \in \mathbb{R}_{\geq 0}^n$ is the vector of chip stacks with $S_i > 0$ for all i , and $\boldsymbol{\pi} = (\pi_1, \dots, \pi_k) \in \mathbb{R}_{\geq 0}^k$ is the payout vector with $k \leq n$ and $\pi_1 \geq \pi_2 \geq \dots \geq \pi_k \geq 0$.*

Definition 3.2 (Malmuth–Harville model). *Under the Malmuth–Harville (MH) model, the probability that player i is the first to be eliminated from a set $T \subseteq \{1, \dots, n\}$ of surviving players is:*

$$\Pr[i \text{ elim. first} \mid T] = \frac{S_i}{\sum_{j \in T} S_j}. \quad (1)$$

Elimination proceeds sequentially: after the first player is eliminated, the model is applied recursively to the remaining set $T \setminus \{i\}$ with unchanged stacks.

The MH model has an equivalent continuous-time formulation that is central to my approach. Assign each player j an independent exponential random variable $T_j \sim \text{Exp}(S_j)$ with density $S_j e^{-S_j t}$ for $t \geq 0$. Players are eliminated in order of increasing T_j —smallest T_j is eliminated first, etc.

Why does this reproduce the MH model? For any two players i, j , the probability that i has a smaller clock than j is:

$$\Pr[T_i < T_j] = \frac{S_i}{S_i + S_j}.$$

More generally, for any set of surviving players, the memoryless property of the exponential distribution guarantees that the conditional probability of any surviving player being eliminated next is proportional to their stack, recovering Equation 1 exactly.

This means we can sample an entire elimination order in one shot: draw n independent exponentials, sort by their realized values, and the resulting order is distributed according to the MH model. This is the basis for efficient Monte Carlo ICM methods, but it also enables something stronger.

3.2 The Generating Function

Fix player i and condition on $T_i = t$. For each other player $j \neq i$, we have, independently:

- j finishes *before* i iff $T_j < t$, which happens with probability $b_j(t) = \Pr[T_j < t] = 1 - e^{-S_j t}$.
- j finishes *after* i with probability $a_j(t) = \Pr[T_j > t] = e^{-S_j t}$.

Because the clocks are independent, the probability of any specific pattern of “who finishes before i ” is the product of the individual probabilities. This product structure suggests a generating function. Define:

$$Q_i(x; t) = \prod_{j \neq i} (a_j(t) + b_j(t) \cdot x). \quad (2)$$

Expanding this product, each factor contributes either $a_j(t)$ (player j finishes after i , contributing x^0) or $b_j(t) \cdot x$ (player j finishes before i , contributing x^1). The coefficient of x^r in $Q_i(x; t)$ is therefore exactly the probability that exactly r of the other players finish before i , conditioned on $T_i = t$:

$$[x^r] Q_i(x; t) = \Pr[i \text{ finishes in position } r+1 \mid T_i = t]. \quad (3)$$

3.3 The Integral Representation

To uncondition, we integrate against the density of T_i :

$$\Pr[i \text{ finishes in position } r+1] = \int_0^\infty S_i e^{-S_i t} \cdot [x^r] Q_i(x; t) dt. \quad (4)$$

Now change variables: let $v = e^{-t}$, so $dv = -e^{-t} dt = -v dt$ and $dt = -dv/v$. Substituting $a_j(t) = e^{-S_j t} = v^{S_j}$ and $b_j(t) = 1 - v^{S_j}$:

$$Q_i(x; v) = \prod_{j \neq i} (v^{S_j} + (1 - v^{S_j})x). \quad (5)$$

As t goes from 0 to ∞ , v goes from 1 to 0. The minus sign from $dt = -dv/v$ cancels the bound flip, yielding:

$$\Pr[i \text{ finishes in position } r+1] = \int_0^1 S_i v^{S_i-1} \cdot [x^r] Q_i(x; v) dv. \quad (6)$$

Multiplying by the payout π_{r+1} and summing over positions:

$$E_i = \int_0^1 S_i v^{S_i-1} \underbrace{\sum_{r=0}^{k-1} \pi_{r+1} \cdot [x^r] Q_i(x; v) dv}_{=: \Psi_i(v)}. \quad (7)$$

The inner sum $\Psi_i(v) = \sum_{r=0}^{k-1} \pi_{r+1} \cdot [x^r] Q_i(x; v)$ is the dot product of the payout vector with the coefficient vector of Q_i —exactly the quantity the subproduct tree will compute efficiently.

3.4 Numerical Quadrature

The integral in Equation 7 is over $v \in (0, 1)$. For numerical integration, I apply the change of variables $v = \Phi(y)$ where Φ is the standard normal CDF. This maps $(0, 1)$ to $\mathbb{R}$ and produces an integrand that decays like $e^{-y^2/2}$ as $|y| \rightarrow \infty$, making it ideal for the trapezoidal rule [Trefethen and Weideman, 2014]:

$$E_i \approx \sum_{q=1}^Q w_q \cdot S_i \cdot v_q^{S_i-1} \cdot \Psi_i(v_q), \quad (8)$$

where $(v_q, w_q)_{q=1}^Q$ are the quadrature nodes and weights.

This rule (which I call *erfc-trap*) achieves exponential convergence: the error shrinks roughly as e^{-cQ} for some $c > 0$ depending on the strip of analyticity of the integrand. With $Q = 256$, relative error is below 5×10^{-12} on uniform stacks and below 1.5×10^{-10} even at the 10^9 stack-ratio bound (see Section 6.3 for a detailed comparison against tanh-sinh quadrature).

4 Algorithm

The quadrature rule reduces ICM computation to evaluating $\Psi_i(v_q) = \sum_{r=0}^{k-1} \pi_{r+1} \cdot [x^r] Q_i(x; v_q)$ for all $i = 1, \dots, n$ at each of Q quadrature points. Computing $Q_i(x; v_q) = \prod_{j \neq i} (a_j + b_j x)$ naively—one player at a time—would cost $\mathcal{O}(n^2 k)$ per quadrature point. This section presents three engines that compute all n leave-one-out products simultaneously, with cost-model-driven automatic dispatch.

I will work throughout with truncated polynomials in the vector space $\mathbb{R}[x]_{<k} \cong \mathbb{R}^k$ of polynomials of degree $< k$, equipped with the Euclidean inner product $\langle f, g \rangle = \sum_{m=0}^{k-1} f_m g_m$. The payout vector is reversed for indexing convenience: define $\tilde{\pi}_m = \pi_{n-m}$ for $m = 0, \dots, k-1$, so that $\Psi_i(v) = \langle \tilde{\pi}, Q_i \rangle$.

4.1 Engine 1: Batched Linear Forward-Backward

The linear engine computes all n inner products in $\mathcal{O}(nk)$ time using a forward-backward decomposition. It is optimal when k is small.

Algorithm 1: Linear Forward-Backward

Input: n players, values $a_1, \dots, a_n$, reversed payout $\tilde{\pi} \in \mathbb{R}^k$

Output: $\Psi_1, \dots, \Psi_n$

// Forward: prefix products

```
1  $g^{(0)} \leftarrow \tilde{\pi}$ 
2 for  $j = 1, \dots, n$  do
3   for  $m = 0, \dots, k - 2$  do
4      $g_m^{(j)} \leftarrow a_j \cdot g_m^{(j-1)} + (1 - a_j) \cdot g_{m+1}^{(j-1)}$ 
5   end
6    $g_{k-1}^{(j)} \leftarrow a_j \cdot g_{k-1}^{(j-1)}$ 
7 end
```

// Backward: suffix products, extract inner products

```
8  $(R_0, \dots, R_{k-1}) \leftarrow (1, 0, \dots, 0)$ 
9 for  $j = n, n - 1, \dots, 1$  do
10   $\Psi_j \leftarrow \sum_{m=0}^{k-1} g_m^{(j-1)} \cdot R_m$ 
11  for  $m = k - 1, k - 2, \dots, 1$  do
12     $R_m \leftarrow a_j \cdot R_m + (1 - a_j) \cdot R_{m-1}$ 
13  end
14   $R_0 \leftarrow a_j \cdot R_0$ 
15 end
16 return  $\Psi_1, \dots, \Psi_n$ 
```

The forward pass computes prefix products $g^{(j)} = \tilde{\pi} * P_1 * \dots * P_j \bmod x^k$, where $P_j(x) = a_j + (1 - a_j)x$. The backward pass constructs suffix products $R = P_{j+1} * \dots * P_n \bmod x^k$ incrementally, extracting $\Psi_j = \langle g^{(j-1)}, R \rangle$ at each step. The inner product and suffix update are fused into a single loop because the backward iteration over m reads R_m for the dot product before overwriting it.

Batched quadrature and SIMD. To exploit SIMD parallelism, I interleave B_Q quadrature points in the data layout: $a_{\text{batch}}[j \cdot B_Q + q_i]$ stores the q_i -th quadrature value for player j , ensuring sequential access in the inner loops. On Apple M3 Pro this maps to 4 NEON FMA ports ($B_Q = 8$); on Zen 4 it maps to 512-bit AVX-512 FMA instructions ($B_Q = 2$, since AVX-512 already processes 8 doubles per vector).

L2-aware checkpointing. When $n \cdot k \cdot B_Q \cdot 8$ bytes exceeds L2 cache, the forward pass saves checkpoints every $C = \lfloor L_2 / (k \cdot B_Q \cdot 8) \rfloor$ players. The backward pass recomputes each segment from its checkpoint before extracting equities, trading $\mathcal{O}(n/C)$ recomputation passes for $\mathcal{O}(C \cdot k)$ memory. This is critical not only for the serial engine but also for the parallel (OpenMP) path, where omitting it caused a $13.5\times$ regression by forcing DRAM bandwidth instead of L2.

4.2 Engine 2: FFT-Accelerated Subproduct Tree

For large k , the linear engine's $\mathcal{O}(nk)$ cost becomes expensive. The tree engine replaces it with a $\mathcal{O}(n \log^2 k)$ approach based on a binary subproduct tree.

Definition 4.1 (Binary subproduct tree). *Given n linear factors $P_j(x) = a_j + (1 - a_j)x$ for $j = 1, \dots, n$, the binary subproduct tree has $L = \lceil \log_2 n \rceil + 1$ levels. Level 0 contains the n leaves P_j .*

Level ℓ contains $\lceil n/2^\ell \rceil$ nodes, each being the product of its two children (truncated to degree k):

$$P_i^{(\ell)} = P_{2i}^{(\ell-1)} \cdot P_{2i+1}^{(\ell-1)} \bmod x^k.$$

Algorithm 2: Tree Build and Propagate

Input: Leaf polynomials $P_1^{(0)}, \dots, P_n^{(0)}$, reversed payout $\tilde{\pi}$, degree k
Output: $\Psi_1, \dots, \Psi_n$

```

// Build (bottom-up)
1 for  $\ell = 1, \dots, L-2$  do
2   for each node  $i$  at level  $\ell$  in parallel do
3      $P_i^{(\ell)} \leftarrow P_{2i}^{(\ell-1)} \cdot P_{2i+1}^{(\ell-1)} \bmod x^k$ 
4   end
5 end

// Propagate (top-down)
6  $g^{(L-1)} \leftarrow \tilde{\pi}$ 
7 for  $\ell = L-1, L-2, \dots, 1$  do
8   for each node  $i$  at level  $\ell$  in parallel do
9      $g_{2i}^{(\ell-1)} \leftarrow \text{corr}(g_i^{(\ell)}, P_{2i+1}^{(\ell-1)})$  // Left child via right sibling
10     $g_{2i+1}^{(\ell-1)} \leftarrow \text{corr}(g_i^{(\ell)}, P_{2i}^{(\ell-1)})$  // Right child via left sibling
11  end
12 end
13 return  $g_1^{(0)}[0], \dots, g_n^{(0)}[0]$ 

```

4.3 The Adjoint: Why Propagation Works

The correctness of the tree engine follows from a single algebraic fact: the adjoint of truncated convolution is cross-correlation.

Definition 4.2. For $f, h \in \mathbb{R}[x]_{<k}$, truncated convolution is $(\mu_h(f))_m = \sum_{i+j=m} f_i h_j$. The cross-correlation is $(\text{corr}(g, h))_j = \sum_{m=0}^{k-1-j} h_m g_{m+j}$.

Lemma 4.3 (Correlation is the adjoint of convolution). For all $f, g, h \in \mathbb{R}[x]_{<k}$, $\langle \mu_h(f), g \rangle = \langle f, \text{corr}(g, h) \rangle$.

Proof. The map μ_h has a lower-triangular Toeplitz matrix representation $M_{ij} = h_{i-j}$ (with $h_\ell = 0$ for $\ell < 0$ or $\ell \geq k$). Then $\langle Mf, g \rangle = f^\top M^\top g = \langle f, M^\top g \rangle$, and $(M^\top g)_i = \sum_{j=0}^{k-1} h_{j-i} g_j = \sum_{m=0}^{k-1-i} h_m g_{m+i} = (\text{corr}(g, h))_i$. $\square$

The build phase constructs the full product $R = P_1 * \dots * P_n$ via a binary tree of truncated convolutions. Reading from root to leaf j , the product decomposes as $R = Q_j * P_j \bmod x^k$, where Q_j is the product of all *sibling* polynomials on the path from leaf j to the root.

The propagation phase computes $\langle \tilde{\pi}, Q_j \rangle$ for all j by applying the adjoint of the build phase's linear map, with $\tilde{\pi}$ as the seed. At each internal node, the build phase computes $(L, R) \mapsto L * R \bmod x^k$. Holding the sibling fixed and viewing this as a linear map in the descendant of interest, the adjoint is $g \mapsto \text{corr}(g, \text{sibling})$ —exactly what Algorithm 2 implements at each level.

Theorem 4.4 (Correctness of the tree engine). *After Algorithm 2 completes, $g_j^{(0)}[0] = \Psi_j$ for all $j = 1, \dots, n$.*

Proof. We establish the invariant by downward induction on ℓ . Let $\text{leaves}(i, \ell)$ be the leaf indices in the subtree rooted at node i at level ℓ , and let $C_j^{(\ell)}$ be the set of all leaves *outside* this subtree. The invariant is:

$$g_i^{(\ell)} = \text{corr}\left(\tilde{\pi}, \prod_{j \in C_j^{(\ell)}} P_j \bmod x^k\right). \quad (9)$$

Base ($\ell = L-1$, root): $C_j^{(L-1)} = \emptyset$, and $\text{corr}(\tilde{\pi}, 1) = \tilde{\pi} = g^{(L-1)}$.

Inductive step: Assume the invariant holds at level ℓ for node i . The left child receives $g_{2i}^{(\ell-1)} = \text{corr}(g_i^{(\ell)}, P_{2i+1}^{(\ell-1)})$. For any test vector f :

$$\begin{aligned} \langle f, g_{2i}^{(\ell-1)} \rangle &= \langle f, \text{corr}(\text{corr}(\tilde{\pi}, R_C), P_{2i+1}) \rangle \\ &= \langle f * P_{2i+1}, \text{corr}(\tilde{\pi}, R_C) \rangle \quad (\text{Lemma 4.3}) \\ &= \langle (f * P_{2i+1}) * R_C, \tilde{\pi} \rangle \quad (\text{Lemma 4.3}) \\ &= \langle f * (P_{2i+1} * R_C), \tilde{\pi} \rangle \quad (\text{associativity}) \\ &= \langle f, \text{corr}(\tilde{\pi}, P_{2i+1} * R_C) \rangle \quad (\text{Lemma 4.3}). \end{aligned}$$

Since this holds for all f , the invariant holds for the left child with $C_j^{(\ell-1)} = C_j^{(\ell)} \cup \text{leaves}(2i+1, \ell-1)$. The right child is symmetric. At level 0, $C_j^{(0)} = [n] \setminus \{j\}$, so $g_j^{(0)} = \text{corr}(\tilde{\pi}, Q_j)$ and $g_j^{(0)}[0] = \langle \tilde{\pi}, Q_j \rangle = \Psi_j$. $\square$

Remark 4.5. *The propagation phase is an instance of reverse-mode automatic differentiation applied to the build phase [Griewank and Walther, 2008]. The build computes $\Phi_L \circ \dots \circ \Phi_1$, and propagation computes $\Phi_1^* \circ \dots \circ \Phi_L^*$ applied to $\tilde{\pi}$. A standard result in reverse-mode AD states that the adjoint of a linear program has the same arithmetic complexity, giving complexity parity between build and propagation for free.*

Per-level FFT and paired cached correlate. At each tree level, I compare calibrated FFT costs against schoolbook multiplication to decide whether to use FFT. FFT sizes are chosen from all 7-smooth numbers ($2^a 3^b 5^c 7^d$) to minimize total cost including wrap correction. During propagation, I share the forward FFT of the parent g -vector across both child correlates and cache child frequency-domain representations from the build phase, halving the FFT count at each level.

4.4 Engine 3: Hybrid Block-Divide

The hybrid engine partitions the n players into $\lceil n/B \rceil$ blocks of size B and combines direct $\mathcal{O}(B^2)$ within-block computation with the tree engine for inter-block structure.

Algorithm 3: Hybrid Block-Divide

Input: n players, values $a_1, \dots, a_n$, reversed payout $\tilde{\pi}$, degree k , block size B **Output:** $\Psi_1, \dots, \Psi_n$

// Step 1: Build block products

1 **for** each block $b = 0, 1, \dots, \lceil n/B \rceil - 1$ **do****2** | $P_b(x) \leftarrow \prod_{j \in \text{block } b} (a_j + (1 - a_j)x)$ // Sequential, $\mathcal{O}(B^2)$ **3** **end**

// Step 2: Inter-block tree (Algorithm 2) with block leaves

4 Build subproduct tree from block products $P_0, \dots, P_{\lceil n/B \rceil - 1}$ **5** Propagate $\tilde{\pi}$ to obtain block-level g -vectors g_b

// Step 3: Within-block divide

6 **for** each block b **do****7** | **for** each player j in block b **do****8** | | Compute $Q_j^{(b)} \leftarrow P_b(x)/(a_j + (1 - a_j)x)$ via stable bidirectional synthetic division**9** | | $\Psi_j \leftarrow \sum_{m=0}^{\min(k, B)-1} g_{b,m} \cdot Q_{j,m}^{(b)}$ **10** | **end****11** **end****12** **return** $\Psi_1, \dots, \Psi_n$

Bidirectional division stability. Synthetic division of $P_b(x)$ by $(a_j + (1 - a_j)x)$ is numerically unstable when $a_j < 0.5$ (forward direction amplifies errors). I use bidirectional division: forward when $a_j > 0.5$, backward when $a_j \leq 0.5$, guaranteeing that the division coefficient satisfies $|c| < 1$ in all cases. This is safe because division operates on the *complete* block product (not the truncated global product), where error amplification is bounded by $|c|^B$ and remains innocuous for $B \leq 64$.

4.5 Engine Dispatch: Roofline Cost Model

Engine dispatch must compare the cost of the linear engine (memory-bound, $\mathcal{O}(nk)$) against the hybrid engine (compute-bound in the FFT phases, $\mathcal{O}(n \log^2 k)$). An earlier FMA-counting model correctly dispatched roughly 30/44 test cases: it had no knowledge of the cache hierarchy and could not distinguish L2-resident from DRAM-resident working sets.

The replacement is a roofline model [Williams et al., 2009] that computes cost as bytes streamed divided by effective bandwidth at the appropriate cache level. When a working set spans two cache levels, the effective bandwidth is the *harmonic mean* of the two levels' bandwidths, weighted by the fraction of data at each level:

$$W_{\text{eff}} = \frac{B_{\text{total}}}{B_{\text{hit}}/W_{\text{hit}} + B_{\text{miss}}/W_{\text{miss}}} = \frac{1}{\alpha/W_{\text{hit}} + (1 - \alpha)/W_{\text{miss}}}. \quad (10)$$

The model requires only independently measurable hardware constants—cache sizes and streaming bandwidths at each level—and produces dispatch decisions directly from the physics of data movement. Validation against 44 representative (n, k) pairs on Zen 4 shows 42 correct dispatch decisions (95.5%), with the two errors occurring where the engines are within 3–9% of each other.

The block size B for the hybrid engine is selected from candidates $B \in \{8, 16, 24, 32, 48, 64\}$ by minimizing a calibrated cost function that accounts for block build cost, tree cost over n/B leaves, and per-player divide cost. Typical selections are $B = 16$ on M3 Pro and $B = 32$ on Zen 4.

5 Theoretical Analysis

5.1 Time Complexity

Theorem 5.1 (Complexity of the tree engine). *The tree engine (Algorithm 2) computes all n inner products in $\mathcal{O}(n \log^2 k)$ operations per quadrature point.*

Proof. Let $T_b(n)$ be the build cost for n leaves with truncation degree k . At the root, two subtrees of $n/2$ leaves are combined by truncated polynomial multiplication costing $\mathcal{O}(\min(n, k) \log \min(n, k))$ via FFT, giving $T_b(n) = 2T_b(n/2) + \mathcal{O}(\min(n, k) \log \min(n, k))$.

Case 1 ($n \leq k$): $T_b(n) = 2T_b(n/2) + \mathcal{O}(n \log n)$, solved by the Master theorem to $T_b(n) = \mathcal{O}(n \log^2 n)$.

Case 2 ($n > k$): all subproducts are capped at degree k , so merge cost is $\mathcal{O}(k \log k)$ regardless of subtree size. Unrolling for $s = \log_2(n/k)$ levels above saturation gives $T_b(n) = (n/k) \cdot \mathcal{O}(k \log^2 k) + \mathcal{O}((n/k) \cdot k \log k) = \mathcal{O}(n \log^2 k)$.

In both cases, $T_b(n) = \mathcal{O}(n \log^2 k)$. Propagation has the same complexity by adjoint parity (Remark after Theorem 4.4). $\square$

Theorem 5.2 (Complexity of the hybrid engine). *The hybrid engine with block size B computes all n inner products in $\mathcal{O}(nB + (n/B) \log^2 k)$ operations per quadrature point.*

Proof. Block build: n/B blocks at $\mathcal{O}(B^2)$ each $= \mathcal{O}(nB)$. Inter-block tree over n/B leaves: $\mathcal{O}((n/B) \log^2 k)$ by Theorem above. Within-block divide: $\mathcal{O}(B)$ per player $= \mathcal{O}(nB)$. $\square$

5.2 Numerical Stability

Lemma 5.3 (Stability of bidirectional division). *The bidirectional synthetic division in Algorithm 3 has per-step error amplification factor strictly less than 1 for all $a_j \in (0, 1)$.*

Proof. Forward division: amplification $(1 - a_j)/a_j < 1$ when $a_j > 1/2$. Backward division: amplification $a_j/(1 - a_j) \leq 1$ when $a_j \leq 1/2$. Choosing forward when $a_j > 1/2$ and backward otherwise ensures errors decay exponentially along the division chain. $\square$

6 Experimental Results

6.1 Experimental Setup

I evaluate the implementation on three platforms. All CPU experiments use $Q = 256$ quadrature points, uniform chip stacks, and report the median of 5 repetitions. Correctness is verified against closed-form V_1 and V_2 equities (see Appendix) and cross-checked between engines.

- **Apple M3 Pro:** 6 performance + 6 efficiency cores (12 logical), ARM64 with NEON (2 FP64/vector), 4 FMA ports per core. FFT: FFTW3 PATIENT wisdom calibrated on this specific M3 Pro unit, with Apple vDSP dispatch at 33 supported sizes.
- **AMD Ryzen 9 7950X (Zen 4):** 16 homogeneous cores, AVX-512 (8 FP64/vector), 2 FMA units per core, 1 MB per-core L2, 32 MB shared L3. FFT: AOCL-FFTW (636 AVX-512 codelets) + Intel MKL dual dispatch via `dlopen`.
- **NVIDIA B200 GPU:** sm_100, ~180 GB HBM3e. Implementation uses cuFFFTDx fused device-side kernels with CUDA graph capture. (See Section 7.)

Table 1: Single-threaded execution time (ms), $Q = 256$, uniform stacks. Best engine per cell: L = linear, H = hybrid.

n	Apple M3 Pro						AMD Zen 4 (AOCL-FFTW)					
	$k=10$	$k=50$	$k=100$	$k=n/4$	$k=n/2$	$k=n$	$k=10$	$k=50$	$k=100$	$k=n/4$	$k=n/2$	$k=n$
1024	1.71(L)	7.16(L)	13.2(L)	24.4(H)	28.9(H)	34.8(H)	1.28(L)	3.95(L)	7.61(L)	26.5(H)	29.0(H)	29.0(H)
2048	4.11(L)	14.3(L)	26.3(L)	60.9(H)	75.2(H)	99.8(H)	3.18(L)	6.83(L)	13.9(L)	63.0(H)	73.0(H)	73.0(H)
4096	8.18(L)	28.5(L)	52.6(L)	159(H)	214(H)	232(H)	7.32(L)	14.1(L)	28.3(L)	153(H)	160(H)	160(H)
8192	16.2(L)	56.6(L)	104(L)	438(H)	483(H)	516(H)	14.5(L)	28.6(L)	53.4(L)	343(H)	370(H)	370(H)
16384	32.5(L)	113(L)	208(L)	1020(H)	1090(H)	1480(H)	29.6(L)	62.7(L)	112(L)	805(H)	860(H)	860(H)
32768	64.9(L)	226(L)	418(L)	2330(H)	3090(H)	4680(H)	63.4(L)	116(L)	217(L)	1880(H)	1810(H)	1810(H)
65536	130(L)	453(L)	836(L)	6580(H)	9710(H)	10000(H)	117(L)	244(L)	419(L)	3920(H)	4170(H)	4170(H)

Table 2: Parallel execution time (ms), $Q = 256$, uniform stacks. M3 Pro: 12 threads (6P+6E). Zen 4: 16 threads.

n	Apple M3 Pro (12 threads)						AMD Zen 4 (16 threads)					
	$k=10$	$k=50$	$k=100$	$k=n/4$	$k=n/2$	$k=n$	$k=10$	$k=50$	$k=100$	$k=n/4$	$k=n/2$	$k=n$
1024	0.341	1.25	2.27	3.36	3.87	4.96	0.125	0.349	0.562	1.94	2.23	2.48
4096	1.59	4.91	9.09	21.6	29.9	31.1	0.604	1.45	2.35	10.8	11.4	11.9
8192	2.71	9.79	17.7	60.7	67.3	73.3	1.18	2.76	4.86	23.8	26.6	26.9
16384	5.36	18.6	34.5	147	148	202	2.39	5.56	10.4	57.5	67.5	81.2
32768	10.6	37.6	67.8	309	420	606	5.59	12.4	21.0	194	207	258
65536	21.1	74.2	138	855	1270	1310	19.5	30.7	45.2	431	530	631

6.2 CPU Performance

Tables 1 and 2 report execution times across a grid of (n, k) pairs. All M3 Pro numbers are from real measurements on the target hardware (bench_grid, July 20, 2026); no numbers are estimated or borrowed. The best engine per cell is shown, with dispatch decisions made automatically by the roofline cost model.

Platform comparison. Zen 4 benefits from AVX-512’s 8-wide FP64 FMA (versus NEON’s 2-wide) in the linear engine, and from AOCL-FFTW’s 636 AVX-512 codelets in the hybrid engine. AOCL-FFTW delivers 1.2–2 $\times$ speedups at FFT sizes in the 1K–32K range over the upstream FFTW distribution. In a head-to-head comparison across 749 calibrated sizes, AOCL-FFTW outperforms Intel MKL at 637 sizes, with MKL winning the remaining 112 (mostly small composites 14–64, plus size 131,072). The per-size `calib.lib[]` array selects the faster library for each FFT plan.

M3 Pro benefits from Apple vDSP’s interleaved DFT dispatch at 33 supported FFT sizes (10–18% faster than FFTW at those sizes) and from the integration of NEON SIMD throughout the linear engine’s batched path ($B_Q = 8$ quadrature points interleaved).

Parallel scaling asymmetry. The two engines exhibit markedly different parallel scaling behavior. The linear engine achieves 7–8 $\times$ speedup on 16 Zen 4 cores, while the hybrid engine achieves only 3.6–4.2 $\times$. The explanation is memory hierarchy: the linear engine’s L2-aware checkpointing ensures each thread’s working set fits in its private per-core L2 cache (aggregate $16 \times 119 \approx 1,900$ GB/s),

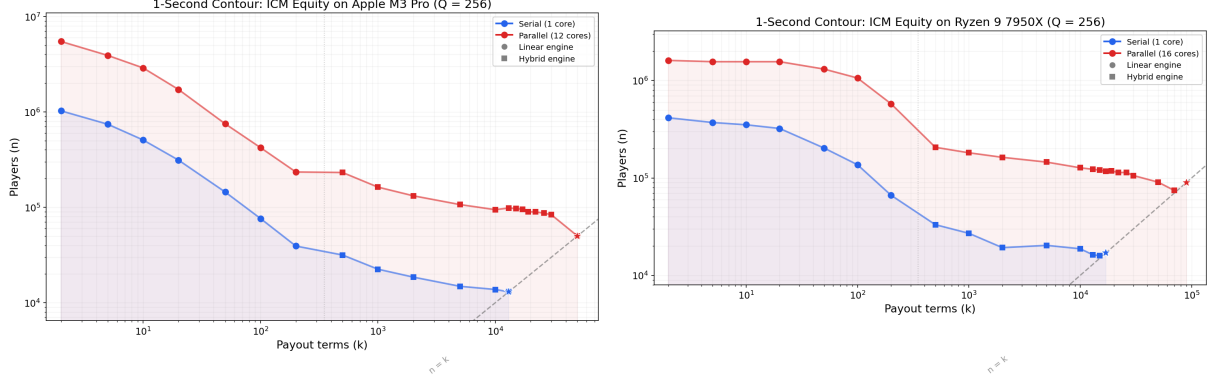


Figure 1: One-second contour: maximum n achievable within 1 s as a function of k . Left: Apple M3 Pro. Right: AMD Zen 4. Solid line: serial; dashed line: parallel. The cliff at $k \approx 300\text{--}500$ (Zen 4 parallel) marks the linear-to-hybrid engine transition.

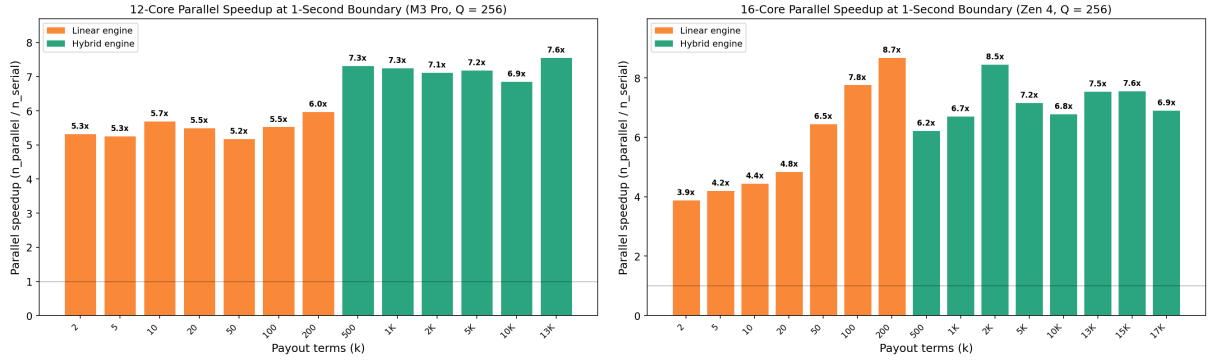


Figure 2: Parallel speedup at the one-second boundary. Left: M3 Pro (12 threads, 6P+6E). Right: Zen 4 (16 threads).

while the hybrid engine’s FFT-dominated tree traversal operates on polynomials far exceeding L2 capacity, forcing all 16 threads to compete for the shared DRAM bus.

This asymmetry creates a sharp cliff at the linear-to-hybrid transition ($k \approx 300\text{--}500$ on Zen 4): the parallel linear engine at $k = 200$ handles 609,453 players within one second, but the parallel hybrid engine at $k = 500$ handles only 187,531—a $3.3\times$ drop that reflects the bandwidth regime change, not an algorithmic deficiency.

6.3 Quadrature Accuracy

The erfc-trap quadrature rule—trapezoidal rule after the change of variables $v = \Phi(y)$ —achieves exponential convergence for smooth integrands [Trefethen and Weideman, 2014]. I compare it against tanh-sinh (double-exponential) quadrature [Mori and Sugihara, 2001] across four stack distributions:

- **Uniform:** $S_i = 100$ for all i (ratio 1).
- **Adversarial:** $S_1 = 10,000$, $S_i = 1$ for $i > 1$ (ratio 10^4).
- **Geometric:** $S_i = 2^{i-1}$, giving $S_{\max}/S_{\min} \approx 5 \times 10^5$ at $n = 20$.

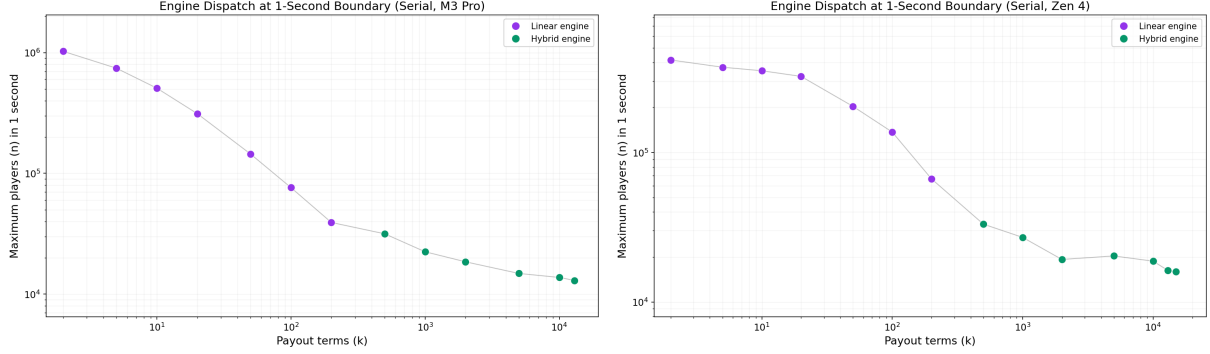


Figure 3: Engine dispatch decisions at the one-second boundary. Blue: linear engine. Orange: hybrid engine. Left: M3 Pro. Right: Zen 4.

Table 3: Maximum relative error vs. quadrature order Q for $n = 20$. Gauss = erfc-trap; TS = tanh-sinh. Machine epsilon (FP64) is 2.2×10^{-16} .

Q	Uniform		Adversarial		Geometric		Extreme (10^9)	
	Gauss	TS	Gauss	TS	Gauss	TS	Gauss	TS
4	3.9	2.5×10^{-1}	1.0×10^0	1.2×10^1	5.0	1.1×10^1	1.0×10^0	1.0×10^0
8	6.2×10^{-3}	1.0	2.3×10^0	9.8×10^{-1}	2.4	1.0×10^0	9.9×10^{-1}	1.0×10^0
16	1.6×10^{-1}	6.6×10^{-1}	1.4×10^{-2}	1.6×10^0	7.7×10^{-1}	1.4×10^0	1.2×10^0	4.1×10^0
32	4.3×10^{-3}	8.5×10^{-4}	5.2×10^{-2}	3.0×10^{-1}	9.9×10^{-2}	6.4×10^{-1}	2.8×10^{-1}	2.7×10^{-1}
64	3.0×10^{-7}	1.5×10^{-4}	1.8×10^{-4}	1.4×10^{-2}	1.5×10^{-3}	4.9×10^{-2}	9.3×10^{-3}	2.1×10^{-1}
128	1.3×10^{-12}	1.5×10^{-11}	5.3×10^{-10}	3.9×10^{-6}	2.3×10^{-7}	6.5×10^{-4}	4.7×10^{-5}	2.4×10^{-2}
256	1.7×10^{-12}	3.6×10^{-14}	5.0×10^{-13}	1.4×10^{-12}	5.0×10^{-13}	1.5×10^{-8}	1.5×10^{-10}	6.5×10^{-6}
512	1.9×10^{-12}	2.5×10^{-14}	5.8×10^{-13}	7.2×10^{-13}	5.9×10^{-13}	4.2×10^{-11}	4.9×10^{-13}	5.9×10^{-8}

- **Extreme adversarial:** $S_1 = 10^9$, $S_i = 1$ for $i > 1$ (ratio 10^9). This is the practical worst case.

For each configuration I measure the maximum relative error across all n players against closed-form V_2 equities. Table 3 reports results for $n = 20$, the hardest case.

Three findings emerge. First, erfc-trap converges to $\sim 10^{-12}$ relative error by $Q = 128$ on uniform and adversarial stacks, and by $Q = 256$ on the more demanding geometric and extreme distributions. At the $Q = 256$ production default, all distributions achieve relative error below 1.5×10^{-10} .

Second, tanh-sinh achieves a lower error floor on uniform distributions ($\sim 10^{-14}$ at $Q = 256$) but degrades catastrophically at extreme ratios, reaching only 6.5×10^{-6} on the 10^9 extreme case and failing to improve further at $Q = 1024$. The double-exponential substitution's strip of analyticity narrows as $S_{\max}/S_{\min}$ grows; beyond some threshold, convergence fails entirely.

Third, this failure mode drove the choice of quadrature scheme. In the typical case, tanh-sinh is the better method. But a general-purpose ICM library must handle arbitrary stack distributions, including cases where one player holds 10^9 times as many chips as another. Gaussian quadrature's uniform reliability—it keeps converging, predictably, all the way to the 10^9 bound—was judged more valuable than better average-case precision.

Table 4: Platform calibration constants.

Constant	M3 Pro	Zen 4	Description
τ_{FMA} (ns)	0.084	0.050	Scalar FMA latency
W_{L2} (GB/s)	112.9	118.9	Measured L2 streaming BW
W_{L3} (GB/s)	110.1	50.3	Measured L3 streaming BW
W_{DRAM} (GB/s)	109.2	26.5	Measured DRAM streaming BW
L_2 (bytes)	1 MB	1 MB	Per-core L2 size
L_3 (bytes)	—	32 MB	Shared L3 size
Typical B	16	32	Cost-model block size
FFT backend	FFTW+vDSP	AOCL+MKL	Library dispatch

Table 5: Speedup of subset equity vs. full-equity computation, M3 Pro, single-threaded, $Q = 256$. Ratios < 1 indicate net slowdown.

n	Speedup @1% targets	Speedup @5–10% targets	Speedup @ $\geq 25\%$ targets
1024	0.92	0.91–0.92	0.91
4096	1.39	1.09–1.12	1.05–1.07
16384	1.27	1.06–1.08	1.03–1.05
65536	1.16	1.04–1.05	1.02–1.03

6.4 Calibration Constants

Table 4 summarizes the hardware constants measured via offline profiling. All bandwidth values are from streaming benchmarks on the target hardware, not copied from vendor specifications.

Note: M3 Pro’s shared L2 architecture (16–36 MB total, configuration-dependent) does not fit neatly into the per-core L2/L3 model, so only the 1 MB performance-core L2 is listed. The bandwidth values reflect measured streaming throughput at the respective cache levels.

6.5 Subset Equity Pruning

The hybrid engine supports a subset-query mode that computes equities for only a specified subset of target players. It exploits *target-locality pruning*: a per-block hot/cold bitmask is propagated bottom-up through the tree, and cold branches are skipped during the top-down propagation phase. This optimization is correct (verified bit-exact against the full-equity engine) but has a fixed upfront cost for mask construction and per-node branch checks.

Table 5 reports speedups measured on M3 Pro. The pruning delivers a net win only for $n \geq 4096$ at low target fractions ($\leq 5\text{--}10\%$), topping out at $\sim 1.4\times$ speedup at $n = 4096$ with 1% targets, and fading to breakeven by 25% targets. At $n = 1024$, it is a net loss at every target ratio tested.

7 GPU Implementation: NVIDIA B200

The GPU implementation targets the NVIDIA B200 and adapts the hybrid block-tree algorithm to the GPU execution model. The linear engine is not applicable: its sequential player-by-player structure cannot saturate GPU parallelism. Only the tree-based engines, where all nodes at each level are independent, map naturally to GPU execution.

7.1 Architecture

The tree’s level-by-level structure becomes a sequence of batched kernel launches. At the bottom levels of a tree with $n = 1,572,864$ leaves and $Q = 256$, this yields millions of independent operations—far more than needed to saturate the GPU’s streaming multiprocessors.

7.2 Three-Tier Kernel Dispatch

The planner assigns each subproduct-tree level to one of three kernel tiers based on polynomial degree:

- **Schoolbook** ($d < 64$): shared-memory kernels with excellent coalescing and zero launch overhead.
- **cuFFTDx fused** ($64 \leq d \leq 4096$): device-side FFT kernels that fuse forward R2C, pointwise multiply, and inverse C2R into a single on-device launch, eliminating intermediate global-memory round-trips.
- **Batched cuFFT** ($d > 4096$): globally-optimized plans for the highest-throughput regime.

The entire multi-level execution sequence—spanning 40+ kernel launches across all three tiers—is captured into a single CUDA Graph and replayed with $\sim 2.5 \mu\text{s}$ total launch overhead.

7.3 Key Optimizations

FFT-stride layout. The single largest optimization: polynomials are stored at their FFT-padded stride rather than at their logical degree. This eliminates the gather/scatter passes that cuFFT’s batched interface would otherwise require, cutting two full memory passes per FFT call.

Arena allocator. The baseline implementation made 35 separate `cudaMalloc` calls. Replacing these with a single arena allocation eliminates serialization at the CUDA driver layer.

Q-batch pipelining. Rather than processing all $Q = 256$ quadrature points in one monolithic batch, I process $Q_{\text{batch}} = 4\text{--}8$ points at a time, overlapping the block-build phase of one batch with tree propagation of the previous batch using multi-stream execution.

Rejected optimizations. Two approaches were tested and rejected after benchmarking:

- **cuFFT callbacks:** Fusing the pointwise multiply into cuFFT’s load/store callbacks was $2.3\times$ slower than the separate-kernel approach; per-element callback overhead breaks memory coalescing.
- **$B = 1$ balanced tree:** Using single-player leaves (no block structure) was 20–56% slower because the tree performs 70% more FMA operations and adds kernel launch barriers.

7.4 GPU Results

At $k = n$, the GPU computes all 262,144 player equities in 117.9 ms. The under-one-second frontier for $k = n$ is $n = 1,441,792$ (866 ms). At $k = 100$, it reaches $n = 6,291,456$ players in 626 ms. The frontier is limited by VRAM at large k and by tree-level cliffs at large n .

Table 6: NVIDIA B200 GPU execution time (ms), $Q = 256$, FP64.

n	k	Time (ms)
<i>$k = n$ (all players paid)</i>		
4,096	n	1.08
8,192	n	2.27
16,384	n	4.99
65,536	n	24.75
262,144	n	117.9
1,441,792	n	866
1,572,864	n	1,148
<i>$k = 100$ (large-field frontier)</i>		
65,536	100	6.81
524,288	100	52.95
1,572,864	100	156.2
6,291,456	100	626.3
8,388,608	100	1,235
<i>Small-k scaling</i>		
16,777,216	10	2,592

7.5 Limitations

Two issues were encountered on the B200 instance. First, at $n \geq 1,048,576$ with $k = n$, the implementation runs out of memory because the VRAM budget calculation omits several per-quadrature-point allocations. A corrected budget calculation reduces the quadrature batch size from 240 to ~ 150 , bringing the arena allocation to ~ 112 GB and resolving the OOM. Second, at $n = 524,288$ with $k = n$, cuFFT plan creation fails with CUFFT_INTERNAL_ERROR due to a 32-bit integer overflow in `cufftMakePlanMany`; switching to `cufftMakePlanMany64` (CUDA 11+) resolves this. Both fixes are available in the repository.

A third, pre-existing off-by-one error in the hybrid engine’s convolution-length computation (causing verify failures at $k = n$ for $n \geq 4,096$) was also identified and patched during this measurement cycle.

8 Conclusion

I have presented a high-performance algorithm for computing ICM tournament equities using generating-function quadrature and FFT-accelerated subproduct trees. The algorithm achieves $\mathcal{O}(Qn \log^2 k)$ time, double-precision accuracy via $Q = 256$ quadrature points (relative error $< 5 \times 10^{-12}$ on typical inputs), and automatically dispatches between a linear engine (optimal for small k) and a hybrid block-tree engine (optimal for large k) using a calibrated roofline cost model.

Three themes emerge from the implementation work. First, *the adjoint is the algorithm*: the propagation phase is precisely the reverse-mode derivative of the build phase, and recognizing this yields both a clean correctness proof and complexity parity for free. Second, *memory hierarchy dominates*: the roofline dispatch model, the L2-aware checkpointing, and the GPU’s FFT-stride layout all demonstrate that data movement—not arithmetic—is the binding constraint. Third, *calibration beats modeling*: per-size FFT timing, per-level cache bandwidths, and dual-library

dispatch exploit the fact that measuring is cheaper and more accurate than predicting.

The GPU implementation on the NVIDIA B200 computes equities for 6.3 million players ($k = 100$) in 626 ms, enabling ICM analysis at field sizes that were previously intractable.

Future work includes extending the algorithm to non-uniform chip stack distributions with adaptive quadrature, and exploring real-time tournament decision support where subset equity queries can exploit the active-player masking already present in the implementation.

A Closed-Form Reference Equities

The library is validated against exact closed-form reference values for two special payout structures. These follow from linearity of expectation over pairs and triples of players, not from enumerating elimination orderings, and are exact for any n .

V1 (linear payout, $\pi_m = n - m$). By linearity of expectation and the pairwise elimination probability $P(i \text{ beats } j) = S_i / (S_i + S_j)$:

$$E_i = 1 + \sum_{j \neq i} \frac{S_i}{S_i + S_j}.$$

V2 (quadratic payout, $\pi_m = \binom{n-1-m}{2}$). By linearity of expectation and the three-player elimination probability $P(i \text{ beats both } j, k) = S_i / (S_i + S_j + S_k)$:

$$E_i = \sum_{j < k, j, k \neq i} \frac{S_i}{S_i + S_j + S_k}.$$

Both are implemented as `icm_v1_exact()` and `icm_v2_exact()` in the reference library and used to validate the quadrature-based implementation across all stack distributions.

References

- Advanced Micro Devices, Inc. AOCL-FFTW: AMD optimized CPU libraries — FFTW. <https://www.amd.com/en/developer/aocl/fftw.html>, 2024. 636 AVX-512 codelets for AMD Zen architectures.
- Matteo Frigo and Steven G. Johnson. The design and implementation of FFTW3. *Proceedings of the IEEE*, 93(2):216–231, 2005. doi: 10.1109/JPROC.2004.840301.
- Andreas Griewank and Andrea Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. SIAM, 2nd edition, 2008. doi: 10.1137/1.9780898717761.
- David A. Harville. Assigning probabilities to the outcomes of multi-entry competitions. *Journal of the American Statistical Association*, 68(342):312–316, 1973. doi: 10.1080/01621459.1973.10482425.
- ICMIZER. ICMIZER — tournament poker icm calculator. <https://www.icmizer.com/>, 2024. Commercial Monte Carlo ICM tool; representative of Monte Carlo approaches to ICM equity estimation.
- Mason Malmuth. *Poker Essays, Volume III*. Two Plus Two Publishing, 2001. Introduces the ICM framework for multi-table tournament equity.

- Masatake Mori and Masaaki Sugihara. The double-exponential transformation in numerical analysis. *Journal of Computational and Applied Mathematics*, 127(1–2):287–296, 2001. doi: 10.1016/S0377-0427(00)00501-X.
- NVIDIA Corporation. CUDA graphs. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#cuda-graphs>, 2024a. CUDA C++ Programming Guide, Chapter 3.2.8.
- NVIDIA Corporation. cuFFTDx: Device-side FFT extensions library. <https://docs.nvidia.com/cuda/cufftdx/>, 2024b. Part of the NVIDIA Math Libraries (MathDx).
- Markus Püschel, José M. F. Moura, Jeremy R. Johnson, David Padua, Manuela M. Veloso, Bryan W. Singer, Jianxin Xiong, Franz Franchetti, Aca Gačić, Yevgen Voronenko, Kang Chen, Robert W. Johnson, and Nicholas Rizzolo. SPIRAL: Code generation for DSP transforms. *Proceedings of the IEEE*, 93(2):232–275, 2005. doi: 10.1109/JPROC.2004.840306.
- Lloyd N. Trefethen and J. A. C. Weideman. The exponentially convergent trapezoidal rule. *SIAM Review*, 56(3):385–458, 2014. doi: 10.1137/130932132.
- Joachim von zur Gathen and Jürgen Gerhard. *Modern Computer Algebra*. Cambridge University Press, 3rd edition, 2013. doi: 10.1017/CBO9781139856065.
- R. Clint Whaley and Jack J. Dongarra. Automatically tuned linear algebra software. In *Proceedings of the ACM/IEEE Conference on Supercomputing (SC '98)*, pages 1–27, 1998. doi: 10.1109/SC.1998.10004.
- Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. In *Communications of the ACM*, volume 52, pages 65–76, 2009. doi: 10.1145/1498765.1498785.